



The AI maintenance engineer for your codebase.

Built on IBM Bob 2.0, Ordo scans a repository, ranks what matters most, and hands the team fix plans, test strategies and pull request summaries.



Technical debt grows quietly, until it doesn't.

01 · Invisible

Nobody sees it pile up

Debt accumulates in code no one is looking at. It surfaces as outages, slow delivery or security incidents, when fixing it costs the most.

02 · Noisy

Reports without priorities

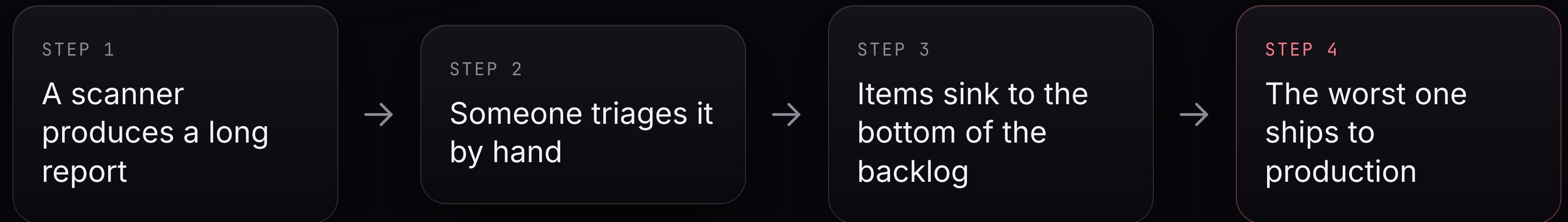
Scanners and linters produce long lists ranked by rule severity, with no sense of business risk or what to do first.

03 · Not actionable

Flagged lines, no fix path

Engineers get findings they cannot act on: no explanation, no plan, no tests, no pull request. So the list gets ignored.

Findings go in. Very little comes out.



The gap is not detection. It is prioritization, explanation and a fix path an engineer can pick up and ship.

Ordo, built on IBM Bob 2.0: from discovery to execution, in one pass.

01

Analyze

Bob reads the repository from GitHub and runs five debt detectors. No cloning, no setup.

02

Prioritize

Bob scores every finding for urgency, business impact and fix effort, with a plain-English reason.

03

Roadmap

Bob sequences the ranked list into a three-phase plan the team can schedule.

04

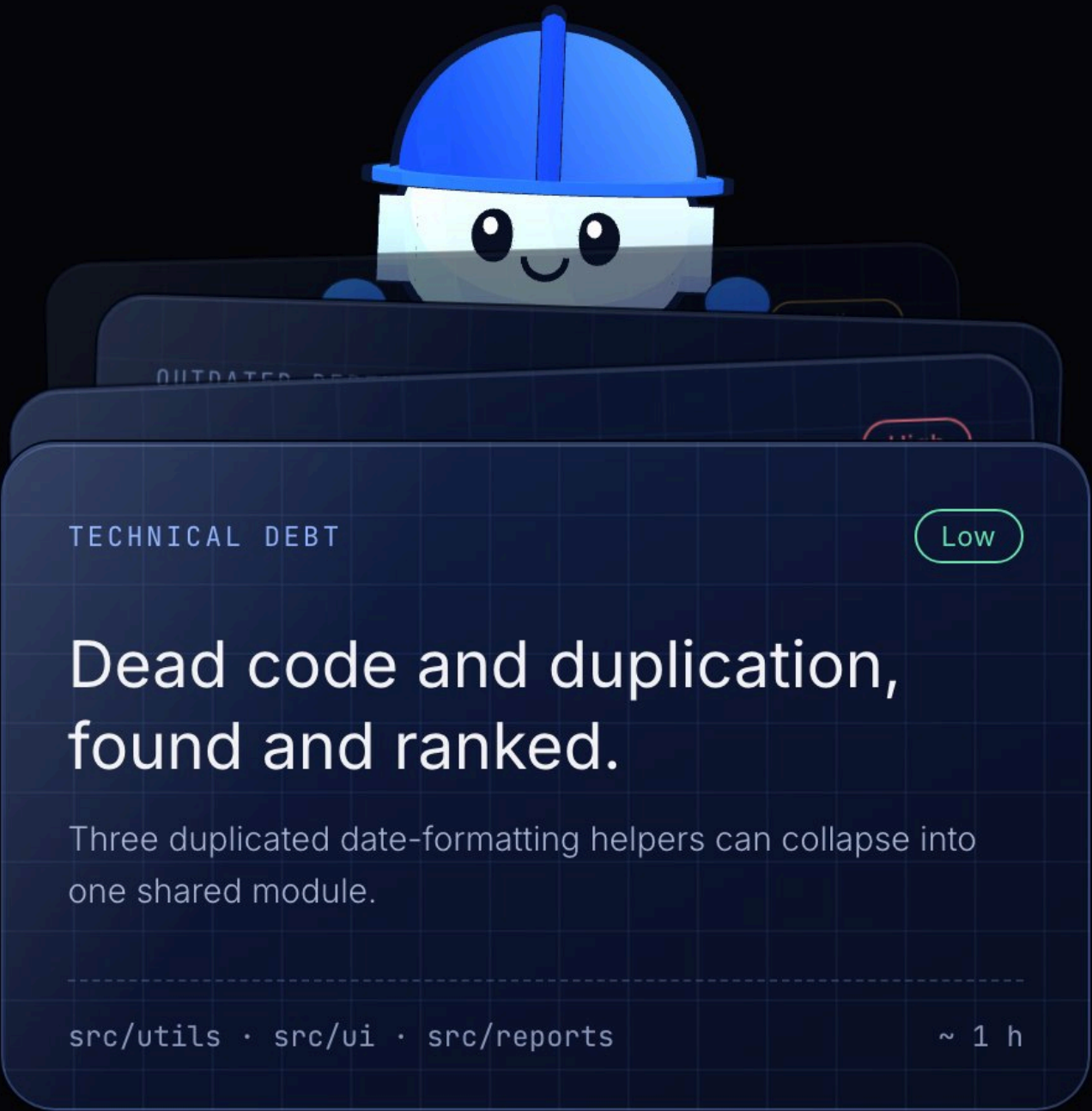
Remediate

Bob prepares the fix plan, the test strategy and a pull request summary, ready to implement.

WHAT BOB FINDS

Five categories of debt that quietly raise costs.

Hardcoded secrets	keys, tokens, passwords
Complex functions	length, nesting depth
Thin test coverage	untested source files
Outdated dependencies	loose or stale versions
Undocumented APIs	exports without docs



TAP A CARD · SAMPLE SCAN

Every finding is ranked by risk, not by rule.

Urgency • 1-10

How soon it needs fixing

Security and reliability risks rise to the top.

Business impact • 1-10

What it costs if ignored

Weighs the damage to users, delivery and trust.

Fix effort • 1-10

What it takes to fix

Surfaces the quick wins next to the big jobs.

IBM Bob 2.0 runs the analysis and does the ranking. IBM Granite on watsonx.ai assists Bob with scoring and a one-line explanation for each finding. If the model is unavailable, Bob falls back to rule-based scores, so a scan never fails.

Bob scanned one of the most-used Node.js frameworks in minutes.

74

FILES ANALYZED OF 214

16

FINDINGS RANKED

15

HIGH URGENCY

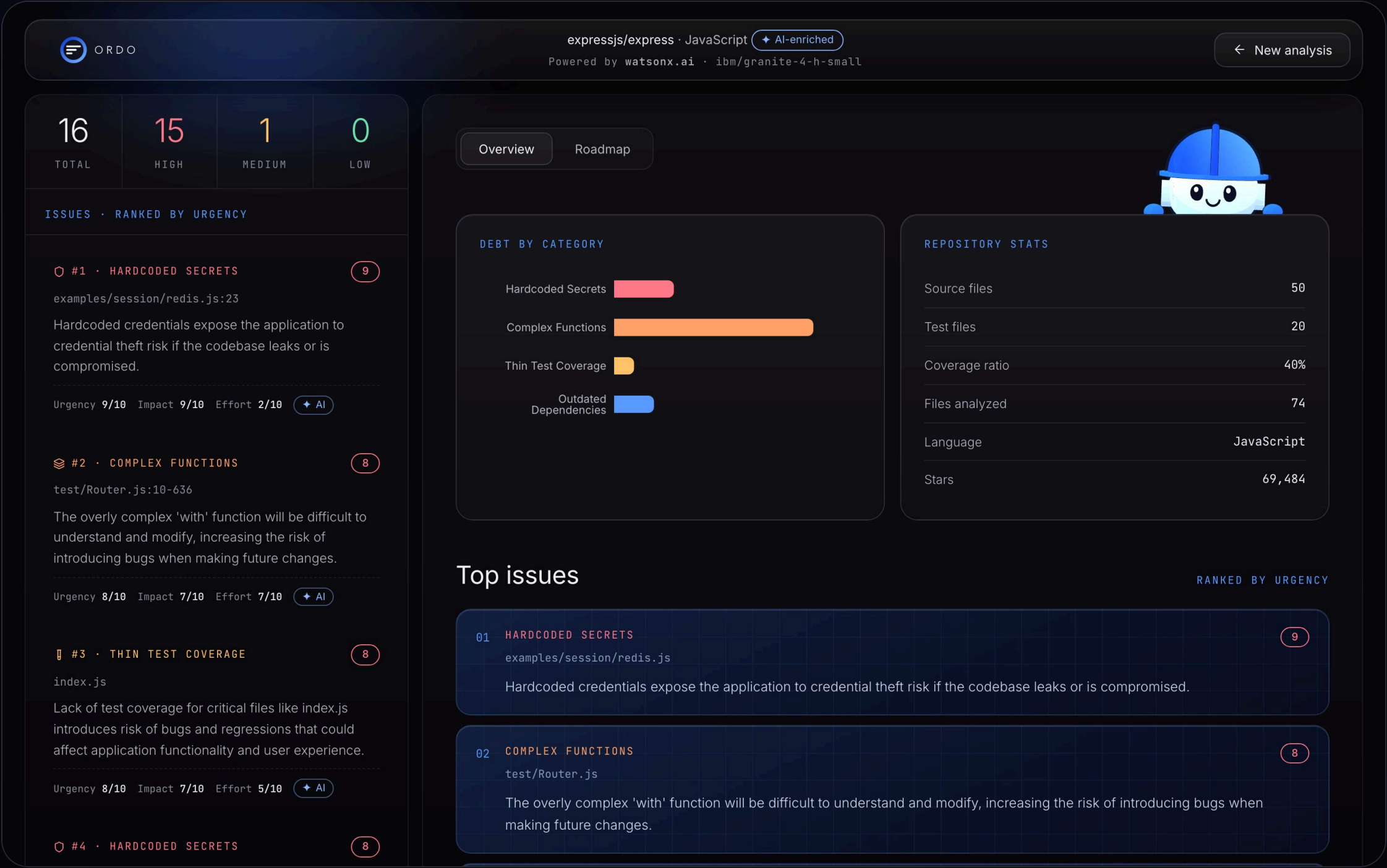
3

PHASE ROADMAP

TOP FINDING	WHERE	URGENCY
HARDCODED SECRET	examples/session/redis.js	9 / 10
COMPLEX FUNCTION	test/Router.js	8 / 10
THIN TEST COVERAGE	index.js	8 / 10

THE PRODUCT

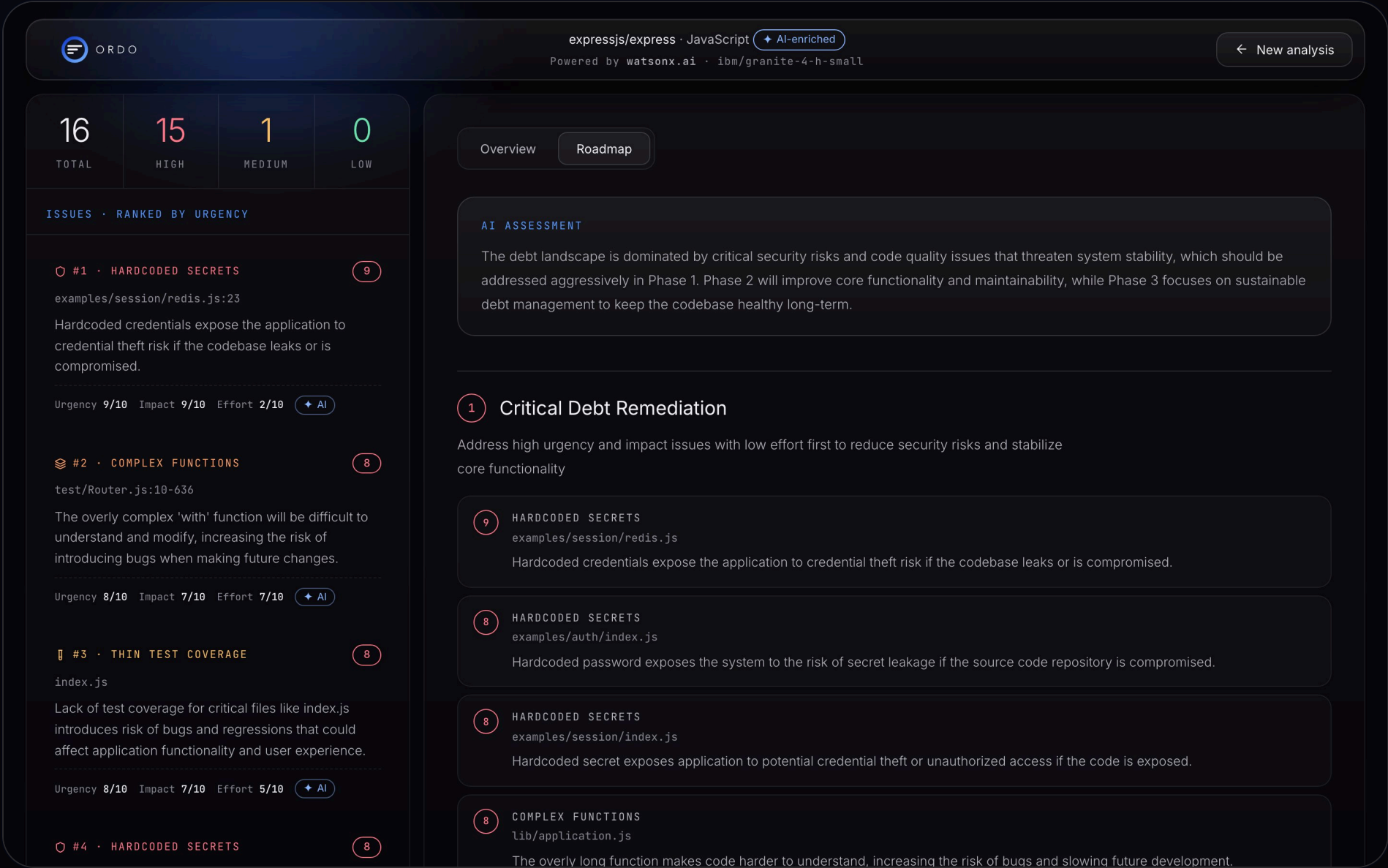
A ranked dashboard, not a wall of warnings.



THE ROADMAP

A maintenance plan the team can schedule.

Ordo sequences the findings into three phases, from fix-now security and reliability issues to longer-term code quality, with the reasoning written out for each phase.



DELIVERABLES

Not just a report. A plan
you can ship.

Remediation plan step by step, with code hints

Test strategy what to verify and how

Pull request summary copy straight into GitHub

ISSUE #1 · HARDCODED SECRETS

```
examples/session/redis.js:23
```

Hardcoded credentials expose the application to data theft risk if the codebase leaks or is compromised.

Urgency 9/10 Impact 9/10 Effort 2/10 AI-score 10

Why is this ranked here?

REMEDIATION PLAN

Remove hardcoded secret from Redis configuration file in redis.js to improve security.

01

Identify the hardcoded secret value at line 23 in examples/session/redis.js.

02

Generate a secure secret using an environment variable or secret management service.

03

Replace the hardcoded secret in the configuration file with the secure secret reference.

04

Update the application to read the secret from the environment variable or secret management service.

05

Remove the hardcoded secret from the code repository.

06

Commit the changes and push to the relevant branch.

Code hints: Use environment variables: process.env. Consider using a secret management service like AWS Secrets Manager or Vault. Ensure proper permission control for the secret.

ISSUE #1 · HARDCODED SECRETS

fix: Remove hardcoded secret from Redis configuration

SUMMARY

This pull request addresses a potential security vulnerability by removing a hardcoded secret found in the Redis configuration file. By replacing the sensitive value with a secure secret referenced from an environment variable or secret management service, we enhance the application's security posture and align with best practices for handling sensitive data.

CHANGES

- [x] Identify hardcoded secret at line 23 in examples/session/redis.js
- [x] Generate a secure secret using environment variable or secret management service
- [x] Replace hardcoded secret with secure secret reference in configuration
- [x] Update application to retrieve secret from environment variable or secret management service
- [x] Remove hardcoded secret from code repository
- [x] Ensure application functions correctly with new secret retrieval method

TESTING

Verify that the application operates as expected with the updated configuration using secure secrets. Confirm that no secrets are inadvertently included in the codebase and that all secret retrieval mechanisms are properly implemented.

Generated by Ordo AI Maintenance Engineer | Powered by watsonx.ai (ibm/granite-3-8b-instruct)

THE EXPERIENCE

Paste a URL. Watch Bob work.

Bob, the face of IBM Bob 2.0 inside Ordo, streams the scan as it runs, then hands the team a dashboard. Every finding leaves as a work order, stamped ready for review.

FREE SCAN

Scan a repository.

Paste a public GitHub repository. Ordo ranks what it finds and prepares the first work order.

REPOSITORY URL

`https://github.com/expressjs/express`

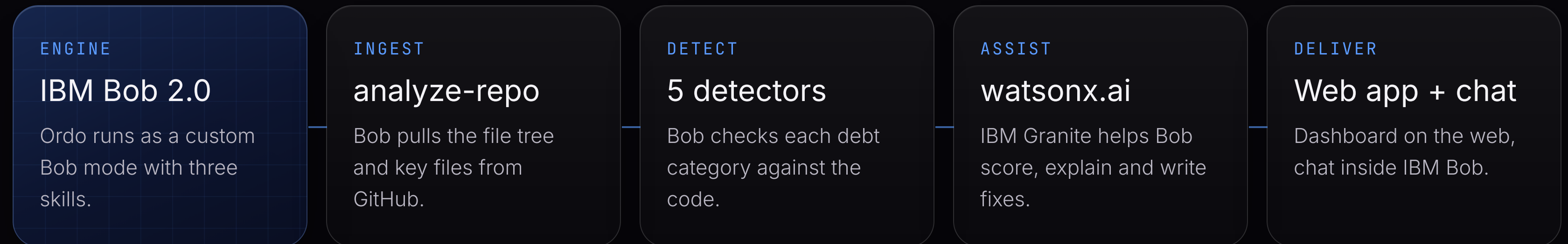
Reading repository and ranking findings... this can take a minute.

```
bob > map imports + 45 edges + 0 cycles
bob > map imports + 45 edges + 0 cycles
bob > check range ^3.2.12 vs latest behind
bob > match /(api[_-]?key|secret|token|passw(or)?d)/i → clean
bob > check range ^7.19.7 vs latest behind
bob > exported symbol #108 docs → missing
bob > check range ^2.12.12 vs latest behind
bob > bob makes a note on his clipboard
bob > exported symbol #179 docs → missing
```

Start Scan →

Close

IBM Bob 2.0 does the work. watsonx.ai assists.



```
you > Analyze https://github.com/expressjs/express
you > Explain issue #3
you > Generate fix for issue #2
```

From a list of problems to a plan of action.

TYPICAL STATIC-ANALYSIS REPORT		ORDO
ENGINE	Static rule set	IBM Bob 2.0, assisted by watsonx.ai
OUTPUT	A long list of findings	A ranked list and a roadmap
PRIORITY	Rule severity	Urgency, business impact and effort
EXPLANATION	Rule identifier	Plain-English reasoning per finding
FIX PATH	Left to the engineer	Fix plan, test strategy, PR summary
INTERFACE	Report file	Web dashboard and IBM Bob 2.0 chat

Transform software chaos into order.

Cut maintenance costs, raise code quality, and stop small issues from becoming production problems.

TRY IT `ordowithbob.vercel.app`

NEXT

Automatic pull requests · CI integration · multi-repo view · debt trends over time · IDE plugin